AireBot: predicción de PM2.5 y agente con RAG para el Valle de Aburrá

SI3003 Inteligencia Artificial, Universidad EAFIT, 2026. Proyecto final integrador. Repositorio: https://github.com/JDavidCE2022/ProyectoFinalIA

1. Problema

En el Valle de Aburrá el SIATA publica a diario el PM2.5 y el Índice de Calidad del Aire (ICA), pero para un ciudadano no es obvio qué significa un valor de 30 µg/m³, si mañana será peor ni qué precauciones tomar. El objetivo fue construir un MVP que (1) prediga el PM2.5 del día siguiente por estación con una técnica clásica implementada por el equipo y (2) lo ponga al alcance de cualquier persona mediante un agente conversacional que consulta datos, usa la predicción y responde con base en la normativa colombiana y en guías de salud, citando sus fuentes.

2. Datos

PM2.5 horario de las 21 estaciones del archivo público del SIATA (`siata.gov.co/EntregaData1/Datos_SIATA_Aire_pm25.json`), del 31 de julio de 2023 al 31 de julio de 2024. Se descartan los datos con indicador de calidad mayor o igual a 2.6, los negativos (faltantes codificados como -9999) y los mayores a 500. El promedio de 24 horas se calcula solo con al menos 18 horas válidas. Quedan 17 estaciones con datos y 5870 días estación. Clima diario de Medellín (lluvia, viento, temperatura, humedad y radiación) de la API de archivo de Open-Meteo, basada en el reanálisis ERA5. Como la API del SIATA solo expone ese año, se trabaja con los históricos descargados (plan B previsto en el enunciado). Solo se usan datos ambientales agregados.

El análisis exploratorio muestra un ciclo diario con pico entre las 7 y las 10 de la mañana y una temporada crítica entre febrero y abril de 2024. El PM2.5 de mañana tiene correlación de 0.82 con el de hoy. Las clases están desbalanceadas: de 5614 casos, 937 son Buena (hasta 12 µg/m³), 4494 Aceptable (13 a 37) y solo 183 Dañina (38 o más, ICA naranja o peor).

3. Método de la Parte 1

Se implementaron dos modelos desde cero en `src/modelo.py`:

Regresión Ridge. Se resuelve por ecuaciones normales, $(A^TA + \lambda I)w = A^T \log(1 + y)$, con $\lambda = 1$ y sin penalizar el intercepto. Las variables son el PM2.5 de hoy y de ayer en la estación, el promedio de la red hoy, lluvia, viento y radiación del día, indicadores de temporada crítica, fin de semana y lunes, y un efecto por estación. El valor predicho se convierte en clase con cortes que se buscan por validación cruzada en bloques dentro del entrenamiento para maximizar el F1 macro. La implementación se verificó contra scikit-learn, con una diferencia máxima de 1e-12.

Naive Bayes categórico. Usa variables discretizadas (clase de hoy, tendencia, lluvia, viento, temporada y día de mañana) con suavizado de Laplace.

Validación temporal. Para cada mes de noviembre de 2023 a julio de 2024 se entrena con todos los días anteriores y se evalúa en ese mes, lo que da 4198 predicciones sin usar información del futuro. Las líneas base son la persistencia (mañana igual a hoy) y la clase mayoritaria.

Modelo	MAE (µg/m³)	Exactitud	F1 macro	Recall Dañina	Precisión Dañina
Ridge + umbrales ajustados	**3.41**	0.804	0.676	**0.692**	0.481
Ridge con cortes del ICA	3.41	**0.832**	0.646	0.522	**0.664**
Naive Bayes		0.801	0.550	0.297	0.562
Persistencia	3.61	0.818	**0.685**	0.604	0.604
Clase mayoritaria		0.786	0.293	0.000	0.000

La Ridge reduce el MAE un 5 % frente a la persistencia, y con los cortes ajustados detecta el 69 % de los días dañinos frente al 60 % de la persistencia, a cambio de más falsas alarmas. En F1 macro la persistencia es ligeramente superior. El análisis por mes explica por qué: la Ridge gana con claridad de abril a julio (por ejemplo, MAE de 2.89 frente a 3.65 en mayo), pero en enero y marzo, cuando el PM2.5 cambia de forma brusca por condiciones meteorológicas del día siguiente, la persistencia es mejor (5.04 frente a 5.43 en marzo). El Naive Bayes queda por debajo de ambas: la discretización pierde información y sus variables no son condicionalmente independientes. Para un uso de salud se eligió la Ridge con cortes ajustados, porque perder un día dañino cuesta más que una falsa alarma.

4. Diseño del agente y del RAG

Loop agéntico (`src/agente.py`). Se implementó a mano con tool calling sobre el endpoint compatible con OpenAI de Groq. Al iniciar consulta los modelos disponibles para la clave y usa el primero de una lista de preferencia; en la evaluación usó `openai/gpt-oss-120b`. El loop tiene un máximo de 5 pasos (Tmáx). Los errores de las herramientas (estación inexistente, fecha sin datos, JSON inválido) vuelven al modelo como observación para que corrija, y ante el error 429 por límite de tokens se espera lo que indica el servidor y se reintenta. El prompt de sistema exige no inventar cifras, citar con el id del fragmento o con [SIATA] y [Modelo del proyecto], abstenerse si no hay información y no dar consejo médico.

Herramientas (`src/herramientas.py`):

- `get_pm25(estacion, fecha, dias)` devuelve la serie medida con la categoría ICA.
- `predict_tomorrow(estacion, fecha)` aplica la Ridge y devuelve el valor con su error típico y, si existe, el valor real observado.
- `search_docs(consulta, k)` busca en el corpus.
- `listar_estaciones()` devuelve las estaciones y el rango de datos.

Todas resuelven la estación por nombre, municipio o serial y nunca lanzan excepciones al agente.

RAG (`src/rag.py`). El corpus tiene 11 documentos (36 fragmentos por párrafo) redactados a partir de la Resolución 2254 de 2017, el POECA del Área Metropolitana, la Alcaldía de Medellín, Cornare y las guías de la OMS 2021. Cada fragmento conserva título, fuente y URL. La búsqueda es híbrida: embeddings de `paraphrase-multilingual-MiniLM-L12-v2` normalizados (el mismo modelo para documentos y consultas) y BM25 con palabras vacías y raíces aproximadas, fusionados con Reciprocal Rank Fusion (k = 60). Si el modelo de embeddings no carga, el sistema sigue funcionando solo con BM25.

Dashboard (`app.py`, Streamlit). Incluye selectores de estación y fecha, indicadores del día y de la predicción, mapa de estaciones coloreado por categoría ICA, serie de tiempo con bandas del ICA y líneas de la norma colombiana y la guía OMS, perfil horario, métricas de validación con la comparación de lo real contra lo predicho, y chat con una tabla de las herramientas usadas en cada respuesta.

5. Evaluación del agente

Se usaron 13 preguntas fijas (`evaluar_agente.py`, resultados completos en `resultados/evaluacion_agente.md`): 3 de datos medidos, 2 de predicción, 5 de documentos, 1 de listado, 1 que combina predicción y documentos, y 1 fuera del alcance (costo del parqueadero del Metro).

Indicador	Resultado
Llamó la herramienta esperada	13/13
Citó la fuente (o se abstuvo en la pregunta fuera de alcance)	13/13
Errores de herramienta	0
Tiempo típico de respuesta	1.6 a 3.4 s (55.6 s en la primera, por la descarga del modelo de embeddings)

En la pregunta combinada el agente encadenó `predict_tomorrow` y `search_docs` y respondió con la categoría predicha y la recomendación correspondiente. En la pregunta fuera de alcance no llamó herramientas y declaró no tener la información.

La métrica automática solo verifica que exista una cita, no que cada afirmación esté respaldada. Una revisión manual encontró dos desviaciones. En la pregunta de asma con ICA naranja el agente añadió consejos que no están en el corpus (purificador de aire, cerrar ventanas), lo que reduce su fidelidad. En la pregunta de Bello dijo "últimos 7 días" pero mostró 5, porque dos días no tenían datos válidos. Una primera corrida falló en 2 preguntas por el límite de 8000 tokens por minuto de la capa gratuita de Groq, lo que llevó a añadir reintentos.

6. Limitaciones

- Hay un solo año de datos y solo 183 casos dañinos, por lo que las métricas de esa clase son inestables.
- El clima viene de un único punto de Medellín para todo el valle y no se usa pronóstico meteorológico del día siguiente, que es lo que limita al modelo en la temporada crítica.
- El corpus es pequeño y redactado a partir de las fuentes, no es el texto legal completo.
- El agente depende de la disponibilidad y los límites de Groq, y sigue pudiendo añadir información general no respaldada.
- No es el pronóstico oficial del SIATA ni consejo médico, y así lo indican la app y el agente.

Como trabajo futuro se propone incorporar el pronóstico de Open-Meteo, más años de datos, un HMM para el estado latente de episodio y una métrica de fidelidad por afirmación.

7. Declaración de uso de IA generativa

Se usó Claude (Anthropic) como asistente de programación y redacción para: descargar las fuentes de datos, proponer la estructura del proyecto, escribir y depurar el código de `src/`, `app.py` y los scripts, redactar los documentos del corpus a partir de las fuentes citadas, y redactar el README y este informe. El autor eligió la idea del proyecto, ejecutó la aplicación y la evaluación del agente en su equipo, detectó los errores de ejecución que llevaron a las correcciones y revisó los resultados, y es responsable de las decisiones finales y de la verificación de que el código corre. Los errores detectados en la ejecución fueron:

- incompatibilidad del SDK de `openai` en Anaconda, que se resolvió llamando a la API con `requests`,
- modelo de Groq no disponible para la clave, que se resolvió con selección automática,
- conteo erróneo de citas en la evaluación,
- límite de tokens por minuto.

La IA también produjo afirmaciones incorrectas que se corrigieron al contrastarlas con los datos, por ejemplo que el modelo mejoraba más en temporada crítica, cuando la tabla por mes muestra lo contrario. El agente del proyecto usa `openai/gpt-oss-120b` vía Groq y embeddings `paraphrase-multilingual-MiniLM-L12-v2`.